MSO 3.7 – Apprentissage bayésien et exploration de textes

Mini-projet : Entraînement et amélioration d'un modèle Transformer (GPT)

Élèves :
Bamishola LOKE
Mamoudou WONE

Enseignant :
Julien Velcin

3 mars 2026

Résumé

Ce rapport présente notre implémentation et évaluation d'un modèle de langage génératif de type GPT, entraîné sur le corpus des *Œuvres complètes de Molière* (1 664 817 caractères). Partant du code fourni en cours (inspiré de nanoGPT, A. Karpathy), nous l'étendons avec des métriques quantitatives, trois stratégies d'amélioration de la génération, et une tokenisation BPE.

Table des matières

1 Mise en place et entraînement **2**
1.1 Corpus et vocabulaire . . . 2
1.2 Architecture myGPT . . . 2
1.3 Convergence . . . 2

2 Évaluations quantitatives **2**

3 Amélioration de la génération **3**
3.1 Stratégie A — Température, Top-k, Nucleus . . . 3
3.2 Stratégie B — Beam Search . . . 3
3.3 Stratégie C — Tokenisation BPE . . . 3

4 Étude des hyperparamètres **4**

5 Tableau comparatif final **4**

6 Conclusion **4**

1 Mise en place et entraînement

1.1 Corpus et vocabulaire

Le corpus est l'intégralité du fichier `moliere.txt` du corpus : 1 664 817 caractères, 114 caractères uniques, découpés en 90 % entraînement (1 498 335 tokens) et 10 % validation (166 482 tokens).

1.2 Architecture myGPT

Nous reprenons le décodeur Transformer du cours (`myGPTV2.py`) et l'augmentons pour tirer parti du GPU Tesla T4 de Google Colab. Les hyperparamètres passent en arguments pour faciliter les comparaisons.

Hyperparamètre	**Valeur initiale**	**Notre valeur**
`n_embd`	32	256
`n_head`	4	8
`n_layer`	3	6
`block_size`	8	256
`batch_size`	4	64
`dropout`	0	0.1
Paramètres totaux	∼30k	**4 858 482**

Table 1 – Comparaison des hyperparamètres

Le dropout (absent du code original) a été ajouté dans les couches `Head` et `FeedForward` pour limiter le surapprentissage. Le checkpointing est implémenté via `torch.save` / `load_state_dict` à chaque évaluation, permettant d'interrompre et reprendre l'entraînement sans perte.

1.3 Convergence

La val loss passe de 4.91 à 1.20 en 5000 itérations et se stabilise après 4000 itérations. L'écart train/val (0.84 vs 1.20) signale un début de surapprentissage maîtrisé grâce au dropout.

2 Évaluations quantitatives

Nous utilisons trois métriques : la **perplexité** $\text{PPL} = \exp(\mathcal{L})$ (nombre de choix hésitants à chaque pas), le **TTR** (ratio mots uniques/total, proche de 1 = texte varié) et le **taux d'hallucination** (mots absents du corpus d'entraînement, vocabulaire de référence : 31 467 mots).

Modèle	**PPL**	**Val Loss**	**TTR**	**Hall.**
Bigramme (baseline)	11.04	2.4011	0.9015	81.1 %
myGPT standard	3.34	1.2074	0.7692	13.9 %

Table 2 – Bigramme vs myGPT

À titre de comparaison, voici un extrait généré par le Bigramme (baseline) :

```
depoiendi   cupa vantse pandie s   PORILUnentrenté [9] ai re lamais e    ête rd'i qusidenerr.
```

Le myGPT réduit la perplexité d'un facteur 3.3 et le taux d'hallucination de 81 % à 14 %. Le TTR du Bigramme est plus élevé (0.90), mais uniquement parce qu'il génère des suites de caractères pseudo-aléatoires formant des «mots» artificiellement variés, sans aucune cohérence lexicale.

3 Amélioration de la génération

3.1 Stratégie A — Température, Top-k, Nucleus

La génération standard échantillonne sur l'ensemble du vocabulaire à chaque pas, ce qui produit parfois des tokens incohérents. Nous implémentons trois mécanismes de contrôle directement dans `generate()` :

Température : les logits sont divisés par T avant le softmax. $T < 1$ concentre les probabilités (plus déterministe) ; $T > 1$ les étale (plus créatif mais moins fiable).

Top-k : seuls les k tokens les plus probables sont conservés, les autres reçoivent une probabilité nulle. Cela supprime les tokens parasites en queue de distribution.

Nucleus (Top-p) : on conserve le plus petit ensemble de tokens dont la somme des probabilités dépasse p. Contrairement au top-k, la taille de l'ensemble s'adapte dynamiquement à la distribution, ce qui le rend plus robuste.

Stratégie	**TTR**	**Hallucination**
Standard ($T = 1.0$)	0.722	5.2 %
Température basse ($T = 0.5$)	0.891	1.8 %
Température haute ($T = 1.5$)	0.969	41.7 %
Top-k ($k = 10$)	0.856	19.2 %
Top-k ($k = 50$)	0.814	18.6 %
Nucleus ($p = 0.9$)	0.816	11.4 %
Nucleus ($p = 0.95$, $T = 0.8$)	0.805	**3.6 %**

Table 3 – Impact des stratégies de sampling (myGPT, PPL fixe = 3.34)

La température haute ($T = 1.5$) maximise la diversité (TTR=0.97) mais explose l'hallucination (41.7 %), produisant des néologismes comme *«ÉRASTEORILLE»* ou *«retouble»*. Le nucleus ($p = 0.95$, $T = 0.8$) offre le meilleur compromis : hallucination de seulement 3.6 % avec une diversité raisonnable (TTR=0.81). C'est la stratégie retenue.

```
Pour moi, par des armes, mon cœur, la maîtresse.
Mon père, qu'on peut tout le monde respirer.

LYCARSIS.

Moi de son des précipiteuses, et l'on s'est dit
Ce qu'il fait juger. Mais enfin que le contre fluchement
Qui puisse me trouver sensible, et que vous doit être si
Pour compliment
```

Figure 1 – Extrait généré par myGPT Nucleus ($p = 0.95$, $T = 0.8$)

3.2 Stratégie B — Beam Search

Le beam search maintient les $w = 5$ meilleures séquences en parallèle selon leur score log-probabilité cumulé. Les métriques semblent excellentes (TTR=1.0, hallucination=0 %), mais l'extrait généré révèle une **dégénérescence** : le modèle produit uniquement des espaces après «ARNOLPHE.». En maximisant la vraisemblance locale, il converge vers des tokens banals — utile en traduction, inadapté à la génération créative.

3.3 Stratégie C — Tokenisation BPE

Nous réentraînons myGPT avec la tokenisation BPE de GPT-2 via `tiktoken` (50 257 sous-mots). L'architecture est allégée pour rester dans les limites mémoire (`n_embd=128`, `n_head=4`, `n_layer=4`, `block_size=64`), pour un total de 13 716 049 paramètres (dominés par la grande table d'embeddings).

Le corpus passe de 1 664 817 tokens caractères à 715 699 tokens BPE, soit une compression de **2.33×**.

Modèle	PPL*	Val Loss	TTR	Hall.
myGPT caractères	3.34	1.2074	0.767	13.9 %
myGPT BPE	21.25	3.0562	0.897	10.3 %

* PPL non comparable entre les deux (vocabulaires différents)

Table 4 – Caractères vs BPE

La PPL BPE (21.25) est mathématiquement plus élevée car le vocabulaire est 440× plus grand, ce qui augmente mécaniquement l'entropie de référence. On juge plutôt la qualité du texte : le TTR BPE (0.90) est nettement supérieur à la version caractères (0.77), et le texte généré contient des mots complets reconnaissables avec des noms de personnages molièresques cohérents (MASCARILLE, SGANARELLE).

4 Étude des hyperparamètres

Configuration	Paramètres	PPL	Val Loss	TTR
Petit (2L, 2H, ctx=64)	118 514	5.60	1.7227	0.941
Moyen (4L, 4H, ctx=128)	837 490	4.13	1.4176	0.905
Grand (6L, 8H, ctx=256)	4 858 482	3.51	1.2542	0.873

Table 5 – Impact de la profondeur et de la taille du contexte (2000 itérations chacun)

Plus le modèle est profond et le contexte large, meilleure est la perplexité. Le TTR diminue légèrement car le grand modèle modélise mieux les structures répétitives du théâtre.

5 Tableau comparatif final

Modèle / Stratégie	PPL	Val Loss	TTR	Hall.
Bigramme (baseline)	11.04	2.4011	0.9015	81.1 %
myGPT standard ($T = 1.0$)	3.34	1.2074	0.7692	13.9 %
myGPT Top-k ($k = 50$, $T = 1.0$)	3.34	1.2061	0.8058	10.8 %
myGPT Nucleus ($p = 0.95$, $T = 0.8$)	3.34	1.2074	0.7737	**3.6 %**
myGPT Beam Search ($w = 5$)	3.34	1.2045	1.0000	0.0 %
myGPT BPE (tiktoken gpt2)†	21.25	3.0562	0.8966	10.3 %

† PPL non comparable (vocabulaire différent)

Table 6 – Comparaison de toutes les configurations

6 Conclusion

Ce projet nous a permis de construire et comprendre un décodeur Transformer de bout en bout. Le **Transformer surpasse largement le Bigramme** (PPL divisée par 3.3, hallucination de 81 % à 14 %) grâce à l'attention multi-têtes. Parmi les stratégies de génération, le **nucleus sampling** ($p = 0.95$, $T = 0.8$) offre le meilleur compromis avec seulement 3.6 % d'hallucination. Le **beam search** dégénère en génération créative malgré des métriques apparemment parfaites. La **tokenisation BPE** améliore la qualité lexicale (TTR=0.90) mais nécessite plus de ressources. Enfin, la **profondeur du modèle** reste le facteur limitant principal : 6 couches donnent PPL=3.51 contre 5.60 pour 2 couches.